UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

MATHEUS CARDOSO PINTO

SUPORTE E AUTOMAÇÃO AO DESENVOLVIMENTO DE JOGOS ELETRÔNICOS

PONTA GROSSA

2026

MATHEUS CARDOSO PINTO

SUPORTE E AUTOMAÇÃO AO DESENVOLVIMENTO DE JOGOS ELETRÔNICOS

Support and Automation for Electronic Game Development

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação, da Universidade Tecnológica Federal do Paraná (UTFPR).
Orientador: André Koscianski

PONTA GROSSA

2026

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

MATHEUS CARDOSO PINTO

SUPORTE E AUTOMAÇÃO AO DESENVOLVIMENTO DE JOGOS ELETRÔNICOS

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação, da Universidade Tecnológica Federal do Paraná (UTFPR).

Data de aprovação: dia/mês por extenso/ano

_____

Nome completo e por extenso do Membro 1 (de acordo com o Currículo Lattes)
Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

_____

Nome completo e por extenso do Membro 2 (de acordo com o Currículo Lattes)
Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

_____

Nome completo e por extenso do Membro 3 (de acordo com o Currículo Lattes)
Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

PONTA GROSSA

2026

RESUMO

A indústria de jogos eletrônicos enfrenta desafios relacionados à crescente complexidade técnica e aos custos envolvidos no desenvolvimento de jogos, especialmente em projetos que demandam grande quantidade de conteúdo. Nesse contexto, a inteligência artificial generativa surge como uma tecnologia capaz de apoiar e automatizar diferentes etapas do desenvolvimento. Este trabalho investiga o uso de inteligência artificial generativa como forma de apoiar o desenvolvimento de jogos eletrônicos, com foco na geração procedural de níveis e salas. Para isso, foi realizada uma revisão da literatura sobre desenvolvimento de jogos, geração procedural de conteúdo e inteligência artificial generativa, seguida da definição de um modelo conceitual para integrar um modelo de linguagem ao processo de geração de níveis. A partir desse modelo, foi desenvolvido um protótipo que recebe descrições de níveis em linguagem natural, utiliza um modelo de linguagem para gerar uma representação estruturada em JSON, realiza a validação dessa representação e utiliza a *engine* Godot para construir e visualizar os níveis gerados. A avaliação foi realizada por meio de 12 experimentos, considerando o tempo de geração, a taxa de sucesso e a validade das estruturas produzidas, além da observação das características dos níveis gerados. Todos os experimentos foram concluídos com sucesso na primeira tentativa, com tempos de geração entre 20,24 e 94,80 segundos e média de 51,42 segundos. Os resultados indicam a viabilidade da abordagem para automatizar parte do processo de geração de níveis, embora a avaliação realizada tenha se concentrado principalmente nos aspectos estruturais das gerações, não contemplando uma análise sistemática de jogabilidade e balanceamento.

Palavras-chave: inteligência artificial generativa; desenvolvimento de jogos; geração procedural de conteúdo; geração de níveis; automação.

ABSTRACT

The electronic game industry faces challenges related to the increasing technical complexity and costs involved in game development, especially in projects that require large amounts of content. In this context, generative artificial intelligence emerges as a technology capable of supporting and automating different stages of development. This work investigates the use of generative artificial intelligence to support electronic game development, focusing on the procedural generation of levels and rooms. To this end, a literature review was conducted on game development, procedural content generation, and generative artificial intelligence, followed by the definition of a conceptual model for integrating a language model into the level generation process. Based on this model, a prototype was developed that receives level descriptions in natural language, uses a language model to generate a structured representation in JSON, validates this representation, and uses the Godot engine to construct and visualize the generated levels. The evaluation was conducted through 12 experiments, considering generation time, success rate, and the validity of the generated structures, as well as observations of the characteristics of the generated levels. All experiments were successfully completed on the first attempt, with generation times ranging from 20.24 to 94.80 seconds and an average of 51.42 seconds. The results indicate the feasibility of the approach for automating part of the level generation process, although the evaluation focused primarily on the structural aspects of the generated levels and did not include a systematic analysis of gameplay and game balance.

Keywords: generative artificial intelligence; game development; procedural content generation; level generation; automation.

SUMÁRIO

1	**INTRODUÇÃO**	**06**
1.1	**Contextualização**	**06**
1.2	**Definição do problema e motivação**	**06**
1.3	**Objetivos**	**07**
1.3.1	Objetivo geral	07
1.3.2	Objetivos específicos	08
1.4	**Justificativa**	**08**
2	**FUNDAMENTAÇÃO TEÓRICA**	**09**
2.1	**Conceitos fundamentais sobre jogos eletrônicos**	**09**
2.2	**Arquiteturas de jogos e principais componentes**	**10**
2.3	**Inteligência artificial generativa**	**11**
3	**METODOLOGIA**	**15**
3.1	**Tipo de pesquisa e abordagem metodológica**	**15**
3.2	**Etapas do desenvolvimento**	**15**
4	**DESENVOLVIMENTO E RESULTADOS**	**17**
4.1	**Modelagem conceitual proposta**	**17**
4.2	**Descrição do protótipo**	**18**
4.3	**Aplicação prática e testes realizados**	**19**
4.4	**Avaliação dos resultados**	**20**
5	**CONCLUSÃO**	**22**
	REFERÊNCIAS	**24**

1 INTRODUÇÃO

1.1 Contextualização

O desenvolvimento de jogos eletrônicos é uma área importante que combina arte, narrativa e tecnologia. Esse mercado tem crescido rapidamente e movimenta bilhões de dólares, mostrando que os jogos não são apenas uma forma de entretenimento, mas também um meio relevante de comunicação interativa.

Criar um jogo envolve muitos elementos diferentes, como gráficos, sistemas de áudio, interface e, em alguns casos, funcionalidades *online*. Por causa disso, o desenvolvimento exige equipes com profissionais de várias áreas, como programadores, artistas e projetistas. Essa complexidade faz com que os projetos sejam caros e levem bastante tempo para serem concluídos.

Nos últimos anos, a inteligência artificial generativa tem surgido como uma ferramenta promissora neste contexto. Ela pode ajudar a automatizar partes do processo de desenvolvimento, como a criação de imagens, sons e até níveis de jogo. Além disso, permite gerar conteúdo de forma mais rápida, o que pode reduzir custos e tempo de produção.

Apesar dessas vantagens, ainda existem desafios na utilização dessas tecnologias, como a integração com as ferramentas já existentes e a qualidade dos resultados gerados. Mesmo assim, a inteligência artificial tem potencial para transformar a forma como jogos são desenvolvidos, tornando os processos mais eficientes e abrindo novas possibilidades criativas.

1.2 Definição do problema e motivação

Para que a inteligência artificial generativa seja aplicada de forma eficiente no desenvolvimento de jogos, é importante entender bem como um jogo é estruturado. Isso inclui conhecer seus diferentes sistemas e como eles se conectam, permitindo identificar onde essas tecnologias podem ser integradas e qual impacto podem causar no processo de desenvolvimento.

O uso de inteligência artificial nesse contexto vai além de apenas utilizar ferramentas prontas. É necessário compreender como os diversos componentes de um jogo funcionam juntos, formando um sistema complexo. Sem esse entendimento, fica difícil aplicar soluções de forma eficiente ou aproveitar todo o potencial dessas tecnologias.

Além disso, ainda há uma falta de modelos claros que orientem como integrar a inteligência artificial generativa ao processo tradicional de desenvolvimento de jogos. Isso dificulta tanto a adoção por parte de desenvolvedores quanto o avanço de pesquisas na área.

Diante desse cenário, este trabalho tem como objetivo investigar como a inteligência artificial generativa pode ser aplicada dentro da estrutura de desenvolvimento de jogos. A proposta é analisar essa relação e organizar um modelo conceitual que ajude a entender onde e como essas técnicas podem ser utilizadas na prática, contribuindo tanto para estudos acadêmicos quanto para aplicações reais.

1.3 Objetivos

1.3.1 Objetivo geral

O objetivo geral deste trabalho é investigar o uso de inteligência artificial generativa para apoiar e automatizar o processo de desenvolvimento de jogos eletrônicos, com foco na geração procedural de níveis e salas.

1.3.2 Objetivos específicos

Para alcançar esse objetivo, foram definidos os seguintes objetivos específicos:

- Realizar uma revisão da literatura sobre arquiteturas de jogos eletrônicos e o uso de inteligência artificial generativa no desenvolvimento;
- Identificar como técnicas de automação podem ser aplicadas nas diferentes etapas do desenvolvimento de jogos, como criação de conteúdo, geração de níveis e testes;
- Analisar ferramentas e *frameworks* que utilizam inteligência artificial generativa, destacando seus pontos fortes e limitações;
- Propor um modelo conceitual que mostre como essas técnicas podem ser integradas ao processo tradicional de desenvolvimento de jogos;
- Desenvolver um protótipo ou estudo de caso que demonstre a aplicação prática dessas técnicas;
- Avaliar os resultados obtidos, considerando fatores como eficiência, qualidade e impacto no processo de desenvolvimento.

1.4 Justificativa

A indústria de jogos eletrônicos enfrenta desafios cada vez maiores relacionados à complexidade e aos custos de produção. Projetos podem levar anos para serem concluídos e exigir grandes investimentos, o que torna importante buscar formas de tornar o desenvolvimento mais eficiente sem perder qualidade.

Nesse contexto, a inteligência artificial generativa surge como uma alternativa promissora para ajudar a otimizar diferentes etapas do processo. Ela pode contribuir para reduzir o tempo de produção, automatizar tarefas repetitivas e apoiar a criação de conteúdo, o que pode beneficiar tanto grandes estúdios quanto desenvolvedores independentes.

Do ponto de vista acadêmico, ainda há poucos estudos que exploram de forma prática o uso dessas tecnologias no desenvolvimento de jogos. Por isso, investigar esse tema pode ajudar a ampliar o conhecimento na área e servir de base para futuras pesquisas.

Na prática, este trabalho também busca oferecer orientações que possam ajudar no uso dessas ferramentas no dia a dia do desenvolvimento. Além disso, ao analisar o impacto da inteligência artificial, é possível entender melhor como o papel dos desenvolvedores pode mudar com o uso dessas tecnologias.

Por fim, a criação de um protótipo permite aplicar as ideias propostas em um contexto real, ajudando a validar os conceitos e identificar tanto benefícios quanto limitações. Dessa forma, o trabalho contribui não apenas com a teoria, mas também com aplicações práticas para a área de jogos eletrônicos.

2 FUNDAMENTAÇÃO TEÓRICA

Para entender como a inteligência artificial generativa pode ser aplicada no desenvolvimento de jogos, é necessário apresentar alguns conceitos básicos que servem de base para este trabalho.

Primeiro, são abordados os conceitos fundamentais dos jogos eletrônicos, incluindo suas estruturas e os princípios que orientam seu *design*.

Em seguida, são apresentados os principais componentes e arquiteturas que permitem a construção desses jogos, como motores e sistemas que organizam seu funcionamento.

Por fim, são discutidos os fundamentos da inteligência artificial generativa e suas aplicações no contexto de jogos, especialmente na geração de conteúdo.

Dessa forma, esta seção é organizada em três partes que, juntas, fornecem o embasamento necessário para o desenvolvimento das propostas apresentadas neste trabalho.

2.1 Conceitos fundamentais sobre jogos eletrônicos

Jogos eletrônicos são sistemas interativos que combinam elementos de arte, narrativa e tecnologia para criar experiências envolventes. Diferente de outras mídias, os jogos exigem participação ativa do jogador, que influencia diretamente o que acontece durante a experiência (SCHELL, 2019).

Para entender melhor como os jogos funcionam, é comum dividi-los em partes menores. Um dos modelos mais utilizados é o MDA (*Mechanics*, *Dynamics*, *Aesthetics*), que organiza os jogos em três componentes: regras, comportamento durante a execução e experiência do jogador (HUNICKE; LEBLANC; ZUBEK, 2004). As regras definem como o jogo funciona, o comportamento surge da interação dessas regras ao longo do tempo, e a experiência está relacionada às emoções e percepções do jogador.

Outro modelo importante é o *Elemental Tetrad*, que considera quatro elementos principais: regras, narrativa, aspectos visuais e tecnologia. Esses elementos são interdependentes e devem ser pensados de forma integrada durante o desenvolvimento do jogo (SCHELL, 2019).

Além desses modelos, existem princípios que ajudam a orientar a criação de jogos. Um dos mais conhecidos é o equilíbrio entre facilidade e desafio, onde o jogo

deve ser simples de entender no início, mas oferecer profundidade ao longo do tempo (GAME DESIGNING, 2025). O uso de recompensas também é importante para manter o interesse do jogador, permitindo acompanhar seu progresso.

Os jogos também podem ser entendidos como ciclos de interação. Nesse ciclo, o jogador realiza uma ação, o sistema processa essa ação, fornece um retorno e, a partir disso, o jogador aprende e toma novas decisões (KOSTER, 2010). Esse processo se repete continuamente durante o jogo.

Por fim, o mundo do jogo pode ser visto como um ambiente completo, que vai além dos aspectos visuais e inclui regras, tempo, ambiente e até dimensões emocionais e éticas (ADAMS, 2014). Compreender esses elementos é importante para identificar como tecnologias, como a inteligência artificial, podem ser aplicadas no desenvolvimento de jogos.

2.2 Arquiteturas de jogos e principais componentes

A arquitetura de um jogo eletrônico define como seus sistemas são organizados e como eles interagem entre si. Em geral, essa estrutura é implementada por meio de um motor de jogo, que funciona como a base para o desenvolvimento e execução do jogo (MEEGLE, 2025).

Os motores de jogo modernos são compostos por vários sistemas que trabalham juntos para criar a experiência do jogador. O motor gráfico é responsável por gerar as imagens na tela, enquanto o motor de física simula interações como movimento, colisões e gravidade. O sistema de áudio cuida dos sons e músicas, e o sistema de entrada processa as ações do jogador. Além disso, sistemas de *script* permitem definir comportamentos e regras do jogo (BYTEPLUS, 2024).

Outro elemento importante é o fluxo de execução do jogo, que ocorre em ciclos contínuos. Nesse ciclo, o jogo recebe as entradas do jogador, atualiza seu estado interno, renderiza a cena e processa o áudio. Esse processo se repete várias vezes por segundo, garantindo que o jogo funcione de forma fluida e responda rapidamente às ações do jogador (ISETTA ENGINE, 2024).

Nos últimos anos, houve uma mudança na forma como jogos são estruturados internamente. Em vez de depender apenas de programação orientada a objetos, muitos sistemas passaram a adotar o padrão *Entity-Component-System* (ECS). Nesse modelo, os elementos do jogo são organizados em entidades, componentes e sistemas (UMLBOARD, 2024).

As entidades representam objetos do jogo, os componentes armazenam dados específicos (como posição ou vida), e os sistemas aplicam regras e comportamentos sobre esses dados. Essa separação entre dados e lógica facilita a organização do código e pode melhorar o desempenho, especialmente em jogos com muitos elementos (ZHORAU, 2024).

A arquitetura ECS alcança benefícios de performance através do aproveitamento da arquitetura de CPU, da estruturação de dados compactamente para beneficiar localidade em padrões de acesso e da paralelização de lógica quando possível. Os componentes do mesmo tipo são armazenados em *arrays* contíguos na memória, permitindo acesso sequencial eficiente e melhor utilização de *cache* (ZHORAU, 2024).

Comparado à programação orientada a objetos tradicional, o ECS apresenta diferenças fundamentais que impactam como as aplicações são projetadas. Por exemplo, no ECS são usados objetos de dados simples e expostos, invés de encapsulamento. Também são separados os dados dos comportamentos.

Esta separação entre dados e lógica permite alinhamento ótimo de dados na memória e acesso sequencial rápido, essencial para aplicações críticas de performance como jogos com mundos 3D grandes consistindo de milhares de objetos (UMLBOARD, 2024).

De forma geral, as arquiteturas modernas de jogos são projetadas para serem modulares. Isso permite que diferentes partes do sistema sejam desenvolvidas e atualizadas de forma independente, além de facilitar a integração de novas tecnologias.

2.3 Inteligência artificial generativa

A inteligência artificial generativa refere-se a sistemas capazes de criar novos conteúdos, como textos, imagens e sons, a partir de dados de treinamento. Esses sistemas aprendem padrões existentes e utilizam esse conhecimento para gerar novos resultados com características semelhantes.

Nos últimos anos, essa área evoluiu rapidamente, principalmente com o avanço de redes neurais profundas e modelos baseados em *transformers*. Exemplos conhecidos incluem modelos de linguagem, como *ChatGPT*, e sistemas de geração de imagens, como *Stable Diffusion* e *DALL-E*.

As aplicações da inteligência artificial generativa são diversas e incluem áreas como arte, escrita, desenvolvimento de software e jogos. No contexto de jogos eletrônicos, essa tecnologia pode ser utilizada para automatizar a criação de conteúdo, como níveis, personagens e elementos visuais, contribuindo para a redução de tempo e custos de produção (JOURNAL OF ELECTRICAL SYSTEMS AND INFORMATION TECHNOLOGY, 2024).

Entre os principais tipos de modelos utilizados, destacam-se os modelos de linguagem e os modelos de difusão. Os modelos de linguagem são voltados para a geração de texto e funcionam prevendo sequências de palavras com base em grandes volumes de dados. Já os modelos de difusão são utilizados principalmente para geração de imagens, a partir de processos que adicionam e removem ruído de forma controlada (IBM, 2024).

No desenvolvimento de jogos, uma das aplicações mais relevantes é a geração procedural de conteúdo com uso de aprendizado de máquina, conhecida como *PCGML*. Essa abordagem permite gerar automaticamente elementos do jogo, como mapas e níveis, a partir de exemplos existentes (SUMMERVILLE et al., 2018).

Os algoritmos revisados em pesquisas recentes enquadram-se predominantemente em três categorias: métodos baseados em busca como a pesquisa de Monte Carlo em árvores focados em otimizações, métodos baseados em aprendizado incluindo aprendizado de máquina tradicional e *deep learning* como redes adversariais generativas e aprendizado de reforço, e outros métodos como funções de ruído e gramáticas generativas (ARXIV, 2024).

Múltiplos métodos de PCGML têm sido explorados na literatura, incluindo redes neurais com arquiteturas como *long short-term memory networks* (LSTM), *autoencoders* e redes convolucionais profundas, modelos de Markov como n-gramas e cadeias de Markov multidimensionais, *clustering* e fatoração matricial. LSTMs são predominantemente utilizados para criar dados sequenciais, como demonstrado por Summerville e Mateas (2016) ao tratar níveis de Mario como strings e usar LSTMs para criar novos níveis. Já as redes adversárias generativas consistem de duas redes, uma geradora e uma discriminadora, que são treinadas iterativamente para permitir à geradora criar conteúdo mais realístico enquanto a discriminadora melhora em distinguir conteúdo gerado de dados reais (MDPI, 2024).

A integração emergente de *large language models* em PCG representa o desenvolvimento mais recente no campo. Trabalhos como MarioGPT de Sudhakaran

et al. (2024) demonstram geração de níveis *text-to-level* através de *large language models*, enquanto sistemas como PANGeA utilizam LLMs para criar NPCs com conversas dinâmicas e consistentes com a narrativa do jogo. LLMs possibilitam não apenas geração de conteúdo funcional como níveis e mapas, mas também geração de código, automação de tarefas de programação e criação de elementos narrativos interativos. A capacidade de LLMs de compreender instruções em linguagem natural e gerar outputs estruturados torna-os particularmente adequados para interfaces de co-criação onde designers humanos especificam requisitos de alto nível que são transformados em conteúdo concreto de jogo.

A aplicação prática de IA generativa no desenvolvimento de jogos apresenta benefícios significativos em termos de escalabilidade e eficiência, podendo gerar vastas quantidades de conteúdo rapidamente, permitindo mundos de jogo escaláveis sem aumentos proporcionais no tempo de desenvolvimento.

Algoritmos específicos têm sido desenvolvidos para diferentes tipos de conteúdo de jogo. Para geração de terrenos, técnicas como ruído de Perlin e algoritmos fractais são utilizados para criar paisagens naturais, frequentemente aumentados com redes neurais que aprendem a partir de dados de terrenos reais. Para geração de níveis, algoritmos como autômatos celulares e colapso da função de onda têm sido empregados, com estudos recentes demonstrando algoritmos híbridos baseados em aprendizado por reforço e geração semi-aleatória focados em experiência de jogo (MDPI, 2022). Para geração de personagens e itens, as redes adversárias generativas têm sido exploradas para criar sprites de *pixel art* e *assets* visuais, enquanto técnicas de aprendizado de máquina são aplicadas para balanceamento de itens e ajuste de parâmetros de jogabilidade.

Apesar do grande potencial, o uso de inteligência artificial generativa ainda apresenta desafios. Entre eles estão a necessidade de dados de qualidade, dificuldades de integração com ferramentas existentes e o controle sobre a qualidade dos resultados gerados. Por isso, sua aplicação prática ainda exige avaliação cuidadosa (MDPI, 2024).

Mesmo com essas limitações, a inteligência artificial generativa tem potencial para transformar o desenvolvimento de jogos, tornando os processos mais rápidos, flexíveis e escaláveis.

A compreensão dos fundamentos teóricos de jogos eletrônicos, das arquiteturas técnicas que os implementam e das capacidades e limitações da

inteligência artificial generativa estabelece a base necessária para investigar aplicações práticas dessas tecnologias. A próxima seção apresentará a metodologia empregada para desenvolvimento e avaliação de um protótipo que demonstra a integração de técnicas de IA generativa em processos específicos de desenvolvimento de jogos.

3 METODOLOGIA

3.1 Tipo de pesquisa e abordagem metodológica

Este trabalho caracteriza-se como uma pesquisa aplicada, pois busca investigar a utilização de inteligência artificial generativa como forma de apoiar e automatizar parte do processo de desenvolvimento de jogos eletrônicos. A pesquisa combina aspectos qualitativos e quantitativos, sendo realizada por meio de revisão da literatura, desenvolvimento de um protótipo e execução de experimentos.

A revisão da literatura foi utilizada para compreender conceitos relacionados ao desenvolvimento de jogos, arquiteturas de *software*, geração procedural de conteúdo e inteligência artificial generativa, servindo como base para a definição da abordagem utilizada no protótipo.

A etapa experimental foi realizada por meio da implementação e execução de um protótipo voltado à geração procedural de níveis. O sistema recebe descrições em linguagem natural, utiliza um modelo de linguagem para gerar uma representação estruturada do nível, realiza a validação dessa representação e, posteriormente, utiliza a *engine* Godot para construir e visualizar a estrutura gerada.

A avaliação dos resultados foi realizada a partir de 12 experimentos, considerando principalmente aspectos quantitativos, como o tempo de geração, a taxa de sucesso das gerações e a validade das estruturas produzidas. Também foram observadas qualitativamente as características dos níveis gerados, como a quantidade e os tipos de salas, a presença de diferentes elementos e a organização dos caminhos entre as salas.

A pesquisa possui caráter exploratório, por buscar investigar as possibilidades de utilização de inteligência artificial generativa na geração de conteúdo para jogos, e caráter experimental, devido ao desenvolvimento e à execução de testes com o protótipo proposto.

3.2 Etapas do desenvolvimento

O desenvolvimento deste trabalho foi organizado em etapas, com o objetivo de estruturar a investigação e a aplicação de técnicas de inteligência artificial generativa no contexto de jogos eletrônicos.

A primeira etapa consistiu na revisão bibliográfica, abordando conceitos fundamentais relacionados a jogos eletrônicos, arquiteturas de sistemas e

inteligência artificial generativa. Essa etapa teve como objetivo estabelecer a base teórica necessária para o desenvolvimento da pesquisa.

Na segunda etapa, foi realizado um mapeamento das possíveis aplicações da inteligência artificial generativa no desenvolvimento de jogos, identificando áreas onde essas técnicas podem contribuir para automação e suporte ao processo de criação.

Em seguida, foi feita a definição do escopo do trabalho, delimitando o foco em dois tipos de aplicação: a geração procedural de níveis e salas, e a geração de itens com balanceamento. Essa delimitação permitiu direcionar os esforços para problemas mais específicos e viáveis dentro do contexto do projeto.

A quarta etapa consiste na proposição de um modelo conceitual que descreve como técnicas de inteligência artificial generativa podem ser integradas ao fluxo de desenvolvimento de jogos. Esse modelo busca organizar as possibilidades identificadas de forma estruturada.

Na quinta etapa, propõe-se o desenvolvimento de um protótipo com o objetivo de demonstrar a aplicação prática das técnicas estudadas. O protótipo deverá implementar ao menos uma das abordagens definidas no escopo, permitindo observar seu funcionamento em um contexto controlado.

Por fim, a última etapa corresponde à avaliação dos resultados, que será realizada com base em critérios quantitativos e qualitativos. Entre os critérios quantitativos, destacam-se o tempo de geração, a taxa de sucesso e a variedade dos resultados. Já os critérios qualitativos incluem aspectos como jogabilidade, equilíbrio e consistência dos elementos gerados.

4 DESENVOLVIMENTO E RESULTADOS

4.1 Modelagem conceitual proposta

Esta seção apresenta o modelo conceitual proposto para a integração de técnicas de inteligência artificial generativa ao processo de desenvolvimento de jogos eletrônicos, com foco na geração procedural de níveis.

O modelo tem como objetivo estruturar a forma como sistemas baseados em inteligência artificial podem atuar como suporte à criação de conteúdo, mantendo o controle do processo por parte do desenvolvedor. Para isso, propõe-se uma abordagem em que a inteligência artificial é utilizada para gerar estruturas de alto nível, enquanto a construção final do conteúdo é realizada por um sistema determinístico.

A arquitetura proposta é composta por três etapas principais: geração, interpretação e construção.

Na etapa de geração, um modelo de linguagem é utilizado para produzir uma representação estruturada do nível a ser criado. Essa representação descreve elementos como salas, tipos de ambiente e conexões entre diferentes partes do mapa. O uso de uma estrutura intermediária, como JSON, permite que o conteúdo gerado seja interpretável e independente da implementação final.

Na etapa de interpretação, o sistema desenvolvido recebe a estrutura gerada pela inteligência artificial e realiza sua validação e organização. Nessa fase, são verificados aspectos como consistência das conexões, presença de elementos obrigatórios (como ponto inicial e final) e integridade dos dados.

Por fim, na etapa de construção, o sistema converte a estrutura interpretada em um nível jogável. No contexto deste trabalho, essa etapa corresponde à geração

de um mapa baseado em grade, no qual salas e conexões são representadas visualmente por meio de *tiles*. Esse processo é realizado dentro do ambiente de desenvolvimento do jogo, permitindo a visualização direta do resultado.

Essa separação em etapas permite combinar a flexibilidade da inteligência artificial na geração de conteúdo com a previsibilidade de algoritmos tradicionais, reduzindo problemas relacionados à inconsistência e falta de controle. Além disso, o uso de uma representação intermediária facilita a integração com diferentes ferramentas e motores de jogo.

O modelo proposto também favorece a avaliação de resultados, uma vez que torna possível analisar separadamente a qualidade da estrutura gerada pela inteligência artificial e a eficácia do sistema de construção. Dessa forma, o processo de geração de níveis pode ser estudado de maneira mais controlada e sistemática.

4.2 Descrição do protótipo

Com base no modelo conceitual proposto, foi definido um protótipo com o objetivo de demonstrar, de forma prática, a integração entre inteligência artificial generativa e geração procedural de níveis.

O protótipo consiste em um sistema capaz de gerar automaticamente a estrutura de um nível no formato de um conjunto de salas interconectadas. Para isso, utiliza-se uma abordagem híbrida, na qual a inteligência artificial é responsável pela geração da estrutura lógica do nível, enquanto o motor de jogo realiza sua interpretação e representação visual.

A arquitetura do protótipo é composta por três componentes principais: o módulo de geração, o módulo de interpretação e o módulo de construção.

O módulo de geração é responsável por produzir a estrutura inicial do nível. Essa etapa pode ser realizada por um modelo de linguagem, que recebe uma descrição em linguagem natural e retorna uma representação estruturada do nível, contendo informações como salas, tipos de ambiente e conexões. O resultado dessa etapa é armazenado em formato JSON, permitindo fácil manipulação e integração com o sistema.

O módulo de interpretação atua como intermediário entre a saída da inteligência artificial e o sistema de construção. Nesse estágio, os dados gerados são analisados e organizados, garantindo sua consistência. São verificados, por

exemplo, a existência de uma sala inicial, a conectividade entre os elementos e a validade das relações definidas.

Por fim, o módulo de construção é responsável por transformar a estrutura interpretada em uma representação visual dentro do motor de jogo. No contexto deste trabalho, essa etapa é implementada utilizando a engine Godot, por meio do uso de um sistema baseado em grade (*TileMap*). As salas e conexões são convertidas em elementos visuais, permitindo a visualização do nível gerado.

O protótipo foi projetado para ser simples e modular, permitindo a substituição ou adaptação de seus componentes. Essa abordagem facilita a experimentação com diferentes formas de geração e possibilita futuras extensões, como a inclusão de novos tipos de conteúdo ou regras de geração.

Além disso, o sistema permite a execução de múltiplas gerações, possibilitando a análise da variedade e consistência dos resultados obtidos. Dessa forma, o protótipo atende ao objetivo de demonstrar a viabilidade da aplicação de inteligência artificial generativa na criação de conteúdo para jogos eletrônicos.

4.3 Aplicação prática e testes realizados

Após a implementação do protótipo, foram realizados experimentos com o objetivo de verificar o funcionamento do processo de geração automática de níveis e o comportamento do sistema diante de diferentes tipos de requisitos. Os testes foram organizados em diferentes cenários, variando a quantidade e a complexidade das características solicitadas para cada nível.

Em cada experimento, uma descrição textual do nível foi fornecida ao modelo de linguagem. A partir dessa descrição, o modelo gerou uma representação estruturada em formato JSON, contendo as salas, seus tipos e as conexões entre elas. O arquivo gerado foi então submetido às validações implementadas em Python, responsáveis por verificar aspectos como a existência de uma única sala inicial e uma única sala de chefe, a identificação das salas, a consistência das conexões e a possibilidade de navegação entre a sala inicial e a sala de chefe.

Os experimentos contemplaram diferentes situações de geração. Inicialmente, foram utilizados cenários mais simples, com poucas salas e tipos de sala básicos. Em seguida, foram acrescentados requisitos específicos, como a presença de chaves e salas trancadas, a existência de dois caminhos entre determinados pontos do nível e a geração de níveis com maior quantidade de salas.

Também foram realizados testes combinando simultaneamente diferentes requisitos, incluindo quantidades específicas de elementos, chaves, salas trancadas e múltiplos caminhos.

Além da variação de requisitos, foram realizados testes com diferentes formas de descrição das mesmas necessidades. Em um dos cenários, os requisitos foram apresentados de maneira semelhante, porém com alterações na forma de escrita, permitindo observar o comportamento do sistema diante de variações na entrada textual. Também foi utilizado um cenário com uma descrição menos estruturada e sem a especificação exata das quantidades, buscando verificar como o modelo se comporta quando recebe requisitos mais ambíguos.

Ao todo, foram realizados 12 experimentos. Para cada execução, foram registrados os resultados das tentativas de geração, as validações realizadas e o tempo necessário para a geração. Quando a saída produzida pelo modelo não atendia aos requisitos definidos, o sistema utilizava as informações obtidas na validação para solicitar uma nova geração, respeitando o limite de tentativas estabelecido. Os resultados dessas execuções foram armazenados em arquivos de *log*, permitindo posteriormente analisar o número de tentativas, as falhas de validação e os tempos de geração.

Após a validação da estrutura gerada, os níveis foram utilizados pelo projeto desenvolvido na Godot para sua representação visual. Dessa forma, foi possível observar a organização das salas e conexões produzidas pelo modelo e verificar a correspondência entre a representação estruturada em JSON e a construção do nível no motor de jogo.

4.4 Avaliação dos resultados

A avaliação do protótipo foi realizada a partir de 12 experimentos de geração de *dungeons*, utilizando diferentes descrições de entrada. Os experimentos buscaram verificar a capacidade do sistema de gerar estruturas com diferentes quantidades de salas e requisitos, incluindo salas de inimigos, tesouros, chaves, áreas trancadas, chefes e diferentes caminhos entre as salas. Para cada experimento foram registrados o número de tentativas, a validade da *dungeon* gerada e o tempo total de geração.

Os resultados obtidos apresentaram 100% de sucesso nos experimentos realizados. Todas as 12 *dungeons* foram consideradas válidas na primeira tentativa,

sem necessidade de utilizar novas tentativas para corrigir a geração. Além disso, nenhum erro foi registrado nos resultados armazenados. Dessa forma, dentro das condições dos testes realizados, o protótipo foi capaz de atender aos requisitos estruturais definidos para cada experimento.

Em relação ao tempo de geração, os experimentos apresentaram valores entre 20,24 segundos e 94,80 segundos, com média de 51,42 segundos. O menor tempo foi observado no primeiro experimento, que solicitava uma *dungeon* simples composta por uma sala inicial, uma sala de inimigos e uma sala de chefe. Já o maior tempo ocorreu no experimento 9, que solicitava uma estrutura com 15 salas, incluindo oito salas de inimigos, três salas de tesouro, uma chave, uma sala trancada e uma sala de chefe.

Os experimentos também demonstraram que o protótipo foi capaz de gerar diferentes estruturas de *dungeon*. Nos primeiros testes foram utilizadas estruturas lineares e simples, enquanto experimentos posteriores incluíram múltiplos caminhos entre a sala inicial e a sala de chefe. O experimento 4, por exemplo, produziu dois caminhos que partiam da sala inicial e se encontravam na sala de chefe. Estruturas semelhantes, porém com maior quantidade de salas e elementos, foram utilizadas nos experimentos 8 e 10.

Outro aspecto observado foi a capacidade de combinar diferentes tipos de elementos em uma mesma *dungeon*. Os experimentos 3, 6, 7, 9, 10 e 12 incluíram elementos como tesouros, chaves e salas trancadas, além das salas de inimigos e do chefe. No experimento 6, por exemplo, foi solicitado também um caminho opcional contendo uma sala de tesouro, enquanto o experimento 10 combinou múltiplos tipos de salas com dois caminhos diferentes até o chefe.

De maneira geral, os resultados indicam que o protótipo conseguiu interpretar descrições com diferentes níveis de complexidade e transformá-las em estruturas de *dungeon* válidas. A ausência de falhas nos 12 experimentos indica um comportamento consistente do processo de geração e validação dentro do conjunto de testes realizados. Entretanto, os resultados não permitem afirmar que o sistema sempre produzirá estruturas válidas ou que as *dungeons* geradas apresentam determinado nível de qualidade de jogabilidade ou balanceamento, uma vez que essas características não foram avaliadas de forma sistemática nos experimentos realizados.

5 CONCLUSÃO

Este trabalho teve como objetivo investigar formas de utilizar inteligência artificial generativa como suporte e ferramenta de automação no desenvolvimento de jogos eletrônicos, com foco na geração procedural de estruturas de níveis. A partir da fundamentação teórica sobre jogos eletrônicos, arquitetura de jogos, inteligência artificial generativa e geração procedural de conteúdo, foi proposto um modelo de integração entre um modelo de linguagem e uma *engine* de desenvolvimento de jogos.

Como resultado do trabalho, foi desenvolvido um protótipo capaz de receber uma descrição textual de uma *dungeon* e utilizar um modelo de linguagem para gerar uma representação estruturada em formato JSON. Essa representação é posteriormente validada por uma aplicação intermediária e utilizada pela *engine* Godot para construir e visualizar o nível. Essa abordagem permitiu separar a geração da estrutura lógica da construção visual do nível, mantendo a integração entre as diferentes partes do sistema.

A avaliação foi realizada por meio de 12 experimentos, utilizando descrições com diferentes requisitos e níveis de complexidade. Todos os experimentos foram concluídos com sucesso e apresentaram uma estrutura válida na primeira tentativa, sem erros registrados. Os tempos de geração variaram entre 20,24 e 94,80 segundos, com média de 51,42 segundos. Os testes também demonstraram a capacidade do protótipo de gerar diferentes quantidades de salas e estruturas, incluindo dungeons lineares, múltiplos caminhos e combinações de elementos como inimigos, tesouros, chaves, áreas trancadas e chefes.

Os resultados indicam que a utilização de modelos de linguagem pode contribuir para a automação de parte do processo de criação de níveis, principalmente na transformação de descrições de alto nível em estruturas que podem ser interpretadas por um jogo. A utilização de uma representação estruturada

intermediária também mostrou-se adequada para separar a geração realizada pela inteligência artificial da construção do nível na engine, permitindo que os componentes sejam modificados de forma independente.

Apesar dos resultados positivos, existem limitações no trabalho. Os experimentos realizados avaliaram principalmente a geração e a validade estrutural das dungeons, não sendo realizada uma avaliação sistemática da qualidade da experiência de jogo, do balanceamento da dificuldade ou da percepção dos jogadores. Além disso, o conjunto de 12 experimentos não é suficiente para determinar o comportamento do sistema em todos os tipos possíveis de dungeons ou situações de geração. Dessa forma, os resultados devem ser entendidos como uma avaliação inicial do protótipo dentro do escopo definido.

Como trabalhos futuros, podem ser realizados experimentos com um número maior de gerações e com estruturas mais variadas, além da inclusão de métricas específicas para avaliar a qualidade dos níveis gerados. Também pode ser investigada a utilização de mecanismos de avaliação automática capazes de analisar aspectos como balanceamento, acessibilidade, distribuição de recompensas e complexidade dos caminhos. Outra possibilidade é ampliar o protótipo para outros tipos de conteúdo de jogos, permitindo investigar a aplicação da inteligência artificial generativa em diferentes etapas do desenvolvimento.

De modo geral, o trabalho demonstrou a viabilidade de utilizar inteligência artificial generativa como parte de um fluxo automatizado para geração de níveis de jogos. O protótipo desenvolvido apresentou resultados consistentes nos experimentos realizados e permitiu observar, na prática, tanto as possibilidades quanto as limitações da utilização de modelos de linguagem nesse contexto.

REFERÊNCIAS

MARKET INSIGHTS STATISTA. Video games - Worldwide. 2023. Disponível em: https://www.statista.com/outlook/dmo/digital-media/video-games/worldwide. Acesso em: 9 set. 2025 **.**

PAIZANIS, G.; SCHONFELD, R.; PAGANO, E.; SCHMIDT, N. Leveling Up for the New Reality. The Gaming Report. Boston Consulting Group, 12 dez. 2024. Disponível em: https://www.bcg.com/publications/2024/leveling-up-new-reality. Acesso em: 10 set. 2025.

GALLOTTA, R.; TODD, G.; ZAMMIT, M.; EARLE, S.; LIAPIS, A.; TOGELIUS, J.; YANNAKAKIS, G. Large Language Models and Games: A Survey and Roadmap. arXiv, 26 fev. 2024. Disponível em: https://arxiv.org/abs/2402.18659. Acesso em: 10 set. 2025.

BEGEMANN, A.; HUTSON, J. Empirical insights into AI-assisted game development: A case study on the integration of generative AI tools in creative pipelines. Metaverse, v. 5, n. 2, p. 2568, jul. 2024. DOI: 10.54517/m.v5i2.2568. Disponível em: https://doi.org/10.54517/m.v5i2.2568. Acesso em: 10 set. 2025.

SCHELL, J. The art of game design: a book of lenses. 3 ed. Boca Raton: CRC Press. 2019.

HUNICKE, R.; LEBLANC, M.; ZUBEK, R. MDA: a formal approach to game design and game research. AAAI Workshop on Challenges in Game AI, 2004. Menlo Park: AAAI Press. 2004.

GAME DESIGNING. The 3 primary game design principles. 2025. Disponível em: https://www.gamedesigning.org/learn/game-design-principles/. Acesso em: 23 nov. 2025.

KOSTER, R. The fundamentals of game design. 2010. Disponível em: https://www.raphkoster.com/2010/10/12/the-fundamentals-of-game-design/. Acesso em: 23 nov. 2025.

ADAMS, E. Fundamentals of game design. 3 ed. Berkeley: New Riders. 2014.

MEEGLE. Game engine architecture. 2025. Disponível em: https://www.meegle.com/en_us/topics/game-engine/game-engine-architecture. Acesso em: 23 nov. 2025.

BYTEPLUS. Game engine architecture. 2024. Disponível em: https://www.byteplus.com/en/topic/110891. Acesso em: 23 nov. 2025.

ISETTA ENGINE. Engine architecture. 2024. Disponível em: https://isetta.io/blogs/engine-architecture/. Acesso em: 23 nov. 2025.

UMLBOARD. The entity-component-system design pattern. 2024. Disponível em: https://www.umlboard.com/design-patterns/entity-component-system.html. Acesso em: 23 nov. 2025.

ZHORAU, A. The road to entity component system. Medium, 8 mar. 2024. Disponível em: https://medium.com/@AZhorau/the-road-to-entity-component-system-20efee617bf9. Acesso em: 23 nov. 2025.

JOURNAL OF ELECTRICAL SYSTEMS AND INFORMATION TECHNOLOGY. Unveiling the evolution of generative AI (GAI): a comprehensive and investigative analysis toward LLM models (2021-2024) and beyond. v. 11, n. 22, 2024. DOI: 10.1186/s43067-024-00145-1.

IBM. What are diffusion models? IBM Think, nov. 2024. Disponível em: https://www.ibm.com/think/topics/diffusion-models. Acesso em: 24 nov. 2025.

SUMMERVILLE, A. et al. Procedural content generation via machine learning (PCGML). IEEE Transactions on Games, v. 10, n. 3, p. 257-270, 2018. DOI: 10.1109/TG.2018.2846639.

SUMMERVILLE, A.; MATEAS, M. Super mario as a string: platformer level generation via lstms. arXiv preprint arXiv:1603.00930, 2016.

ARXIV. Procedural content generation in games: a survey with insights on emerging LLM integration. 2024. Disponível em: https://arxiv.org/html/2410.15644v1. Acesso em: 24 nov. 2025.

MDPI. Creating a newer and improved procedural content generation (PCG) algorithm with minimal human intervention for computer gaming development. Computers, v. 13, n. 11, p. 304, 2024. DOI: 10.3390/computers13110304.

MDPI. IORand: a procedural videogame level generator based on a hybrid PCG algorithm. Applied Sciences, v. 12, n. 8, p. 3792, 2022. DOI: 10.3390/app12083792.

SUDHAKARAN, S. et al. Mariogpt: open-ended text2level generation through large language models. Advances in Neural Information Processing Systems, v. 36, 2024.